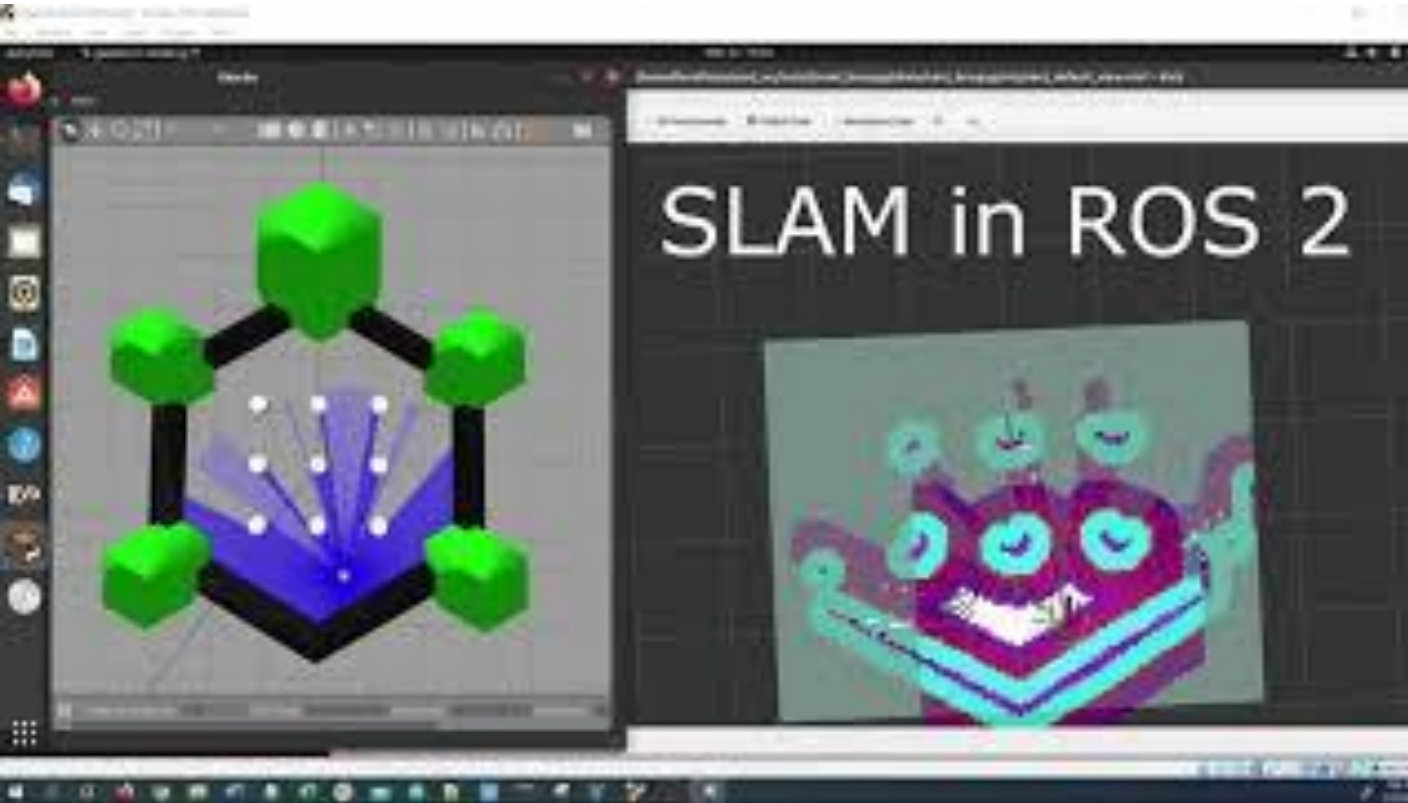


Mobile Robotics

Bachelor's
Degree in
Robotic
Intelligence

Navigation



Navigation with TurtleBot 3 in Gazebo

Terminal 1 (simulation nodes):

```
source /opt/ros/humble/setup.bash
export ROS_LOCALHOST_ONLY=1
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_gazebo turtlebot3_house.launch.py
```

Terminal 2 (navigation nodes):

```
source /opt/ros/humble/setup.bash
export ROS_LOCALHOST_ONLY=1
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_navigation2 navigation2.launch.py \
  use_sim_time:=True map:=<path_to>/map.yaml
```

Required file

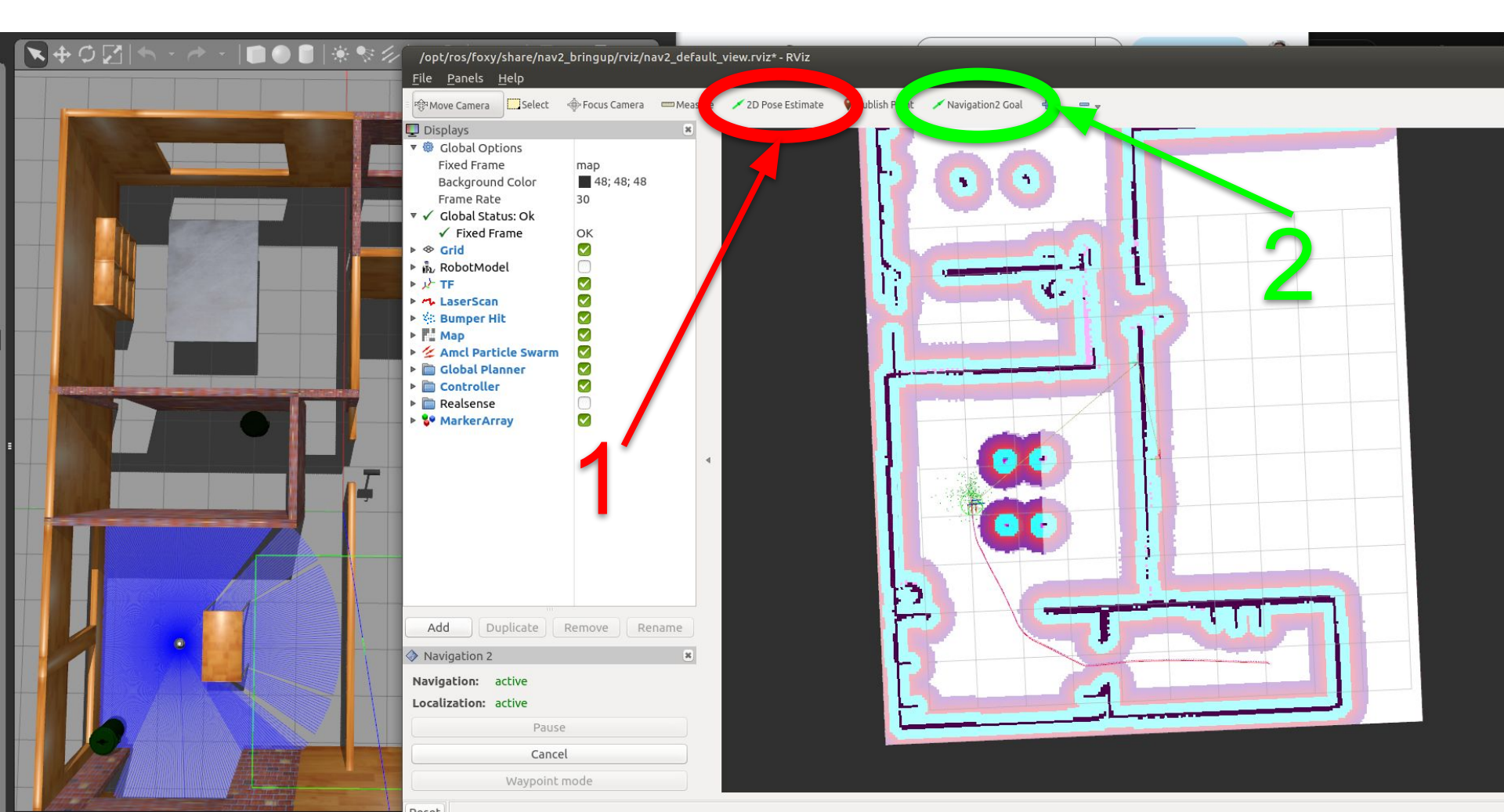
You need to have **a map of the environment** where you navigate.

Required file:

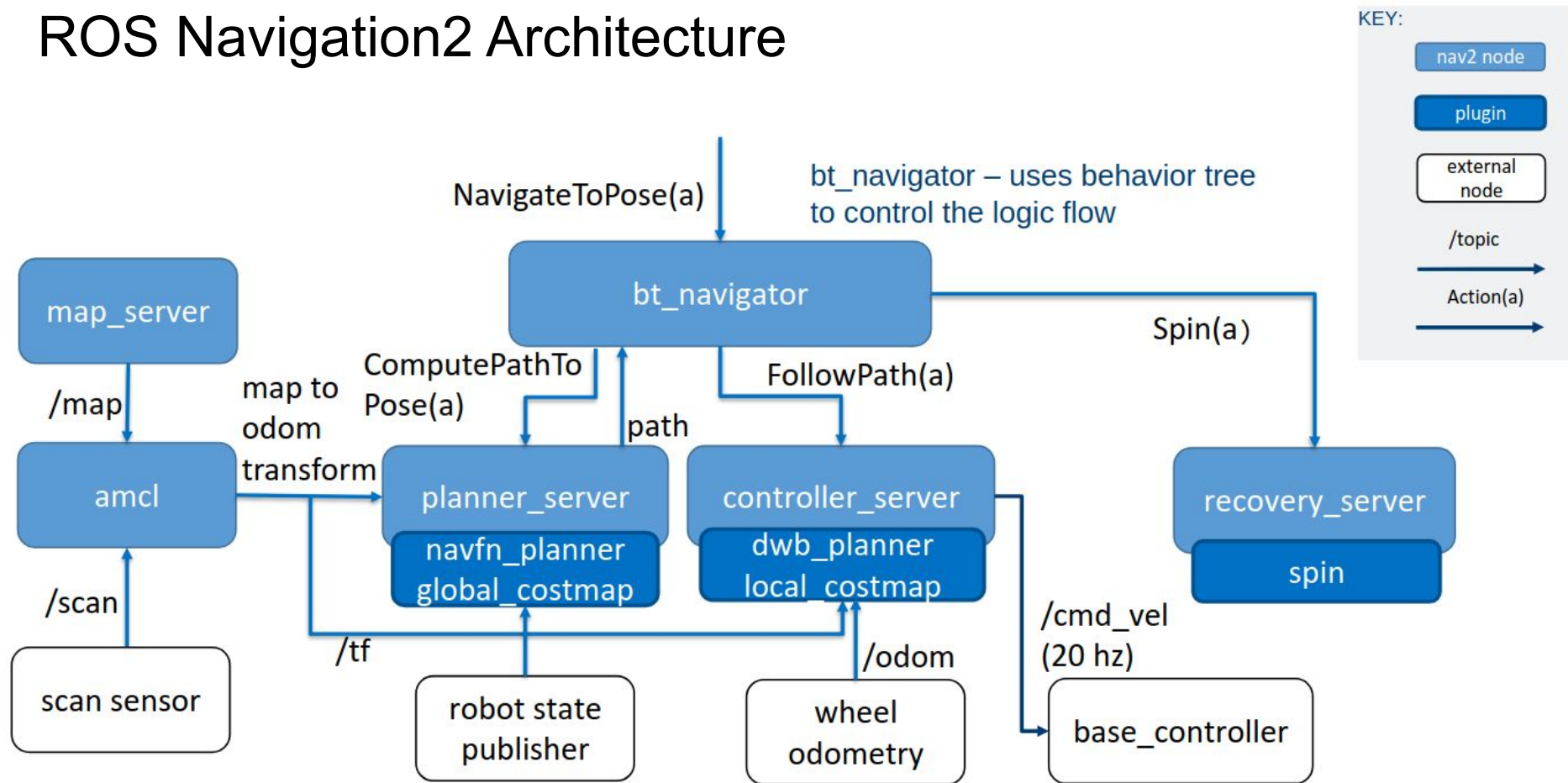
`map.yaml`

`map.yaml` is the configuration file for the map we want to provide Nav2. It has the map resolution value, threshold values for obstacles and free spaces, and a map image file location.

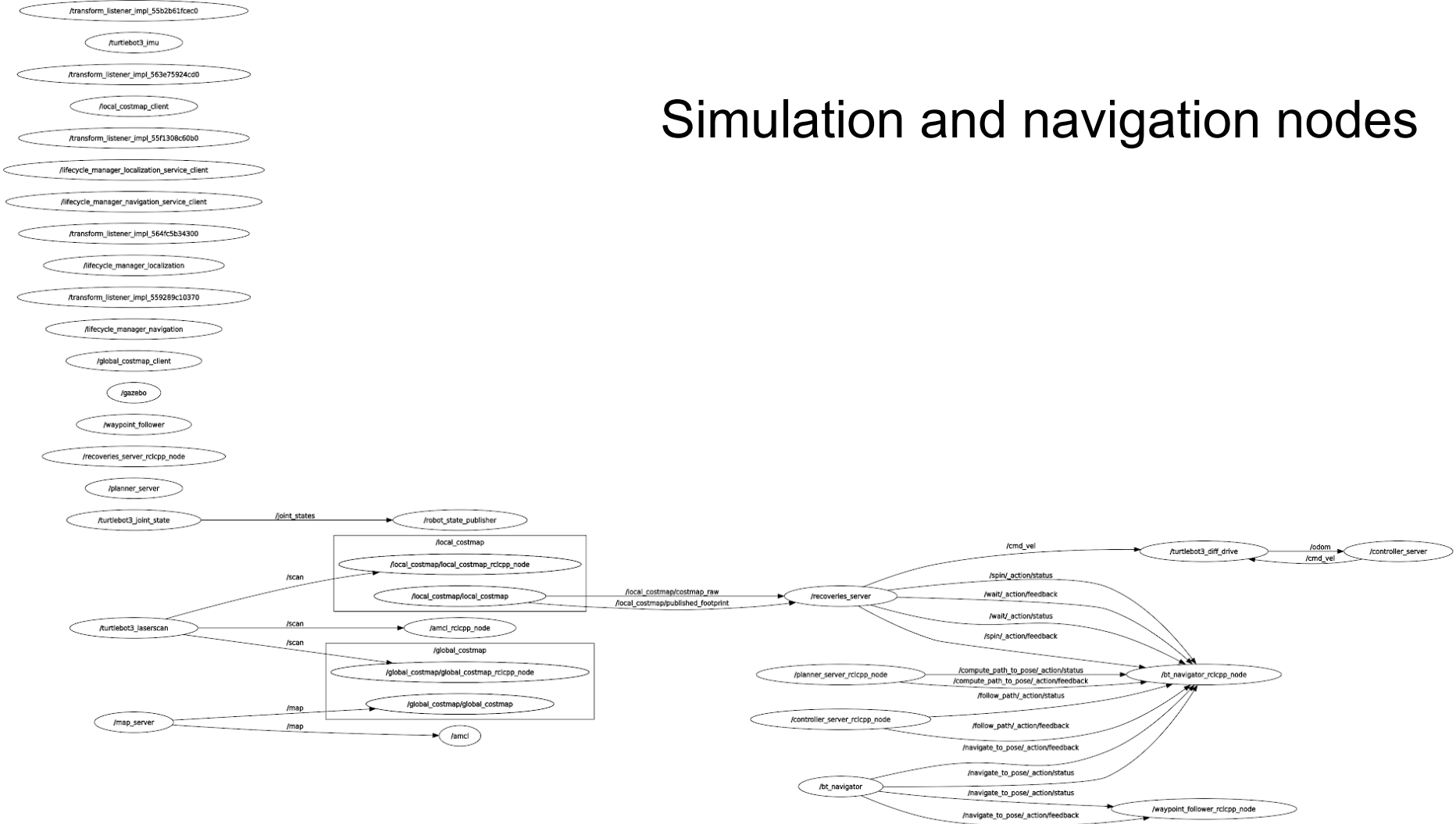
Don't forget to change `<path_to>/map.yaml` to the actual path to the `map.yaml` file.



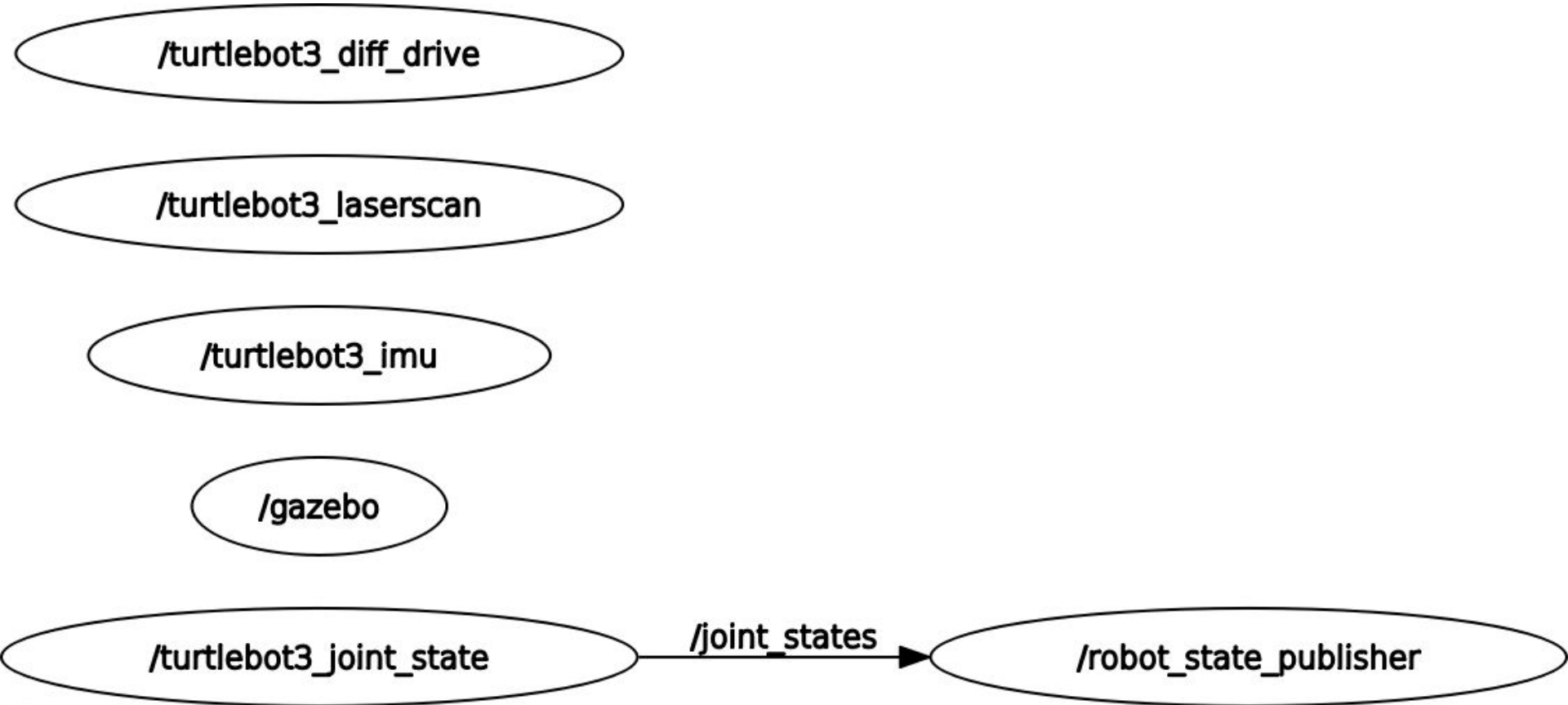
ROS Navigation2 Architecture



Simulation and navigation nodes



Simulation nodes



Simulation nodes

/gazebo

/robot_state_publisher

/turtlebot3_diff_drive

/turtlebot3_imu

/turtlebot3_joint_state

/turtlebot3_laserscan

/gazebo

Simulation engine: ROS 2 Humble (LTS) + GZ Fortress (LTS)

Launchfile:

```
.../turtlebot3_house.launch.py
```

World file:

```
.../worlds/turtlebot3_houses/burger.model
```

Models:

```
.../models/turtlebot3_house/model.sdf
```

```
.../models/turtlebot3_burger/model.sdf
```

Gazebo plugins

`.../models/turtlebot3_burger/model.sdf`

Name	Filename
<code>turtlebot3_imu</code>	<code>libgazebo_ros_imu_sensor.so</code>
<code>turtlebot3_laserscan</code>	<code>libgazebo_ros_ray_sensor.so</code>
<code>turtlebot3_diff_drive</code>	<code>libgazebo_ros_diff_drive.so</code>
<code>turtlebot3_joint_state</code>	<code>libgazebo_ros_joint_state_publisher.so</code>

/robot_state_publisher

Robot State Publisher is a node and a class to publish the state of a robot to tf2.

Once the state gets published, it is available to all components in the system that also use tf2.

The package takes the joint angles of the robot as input and publishes the 3D poses of the robot links, using a kinematic tree model of the robot.

/robot_state_publisher

Parameters

- `robot_description` (string) - The original description of the robot in URDF form. This must be set at `robot_state_publisher` startup time, or the node will fail to start. Updates to this parameter will be reflected in the `robot_description` topic.
- `publish_frequency` (double) - The frequency at which fixed transforms will be republished to the network. Defaults to 50.0.
- `use_tf_static` (bool) - Whether to publish fixed joints on the static broadcaster (`/tf_static` topic) or on the dynamic one (`/tf` topic). Defaults to true, so it publishes on the `/tf_static` topic.
- ...

/robot_state_publisher

Subscribed Topics

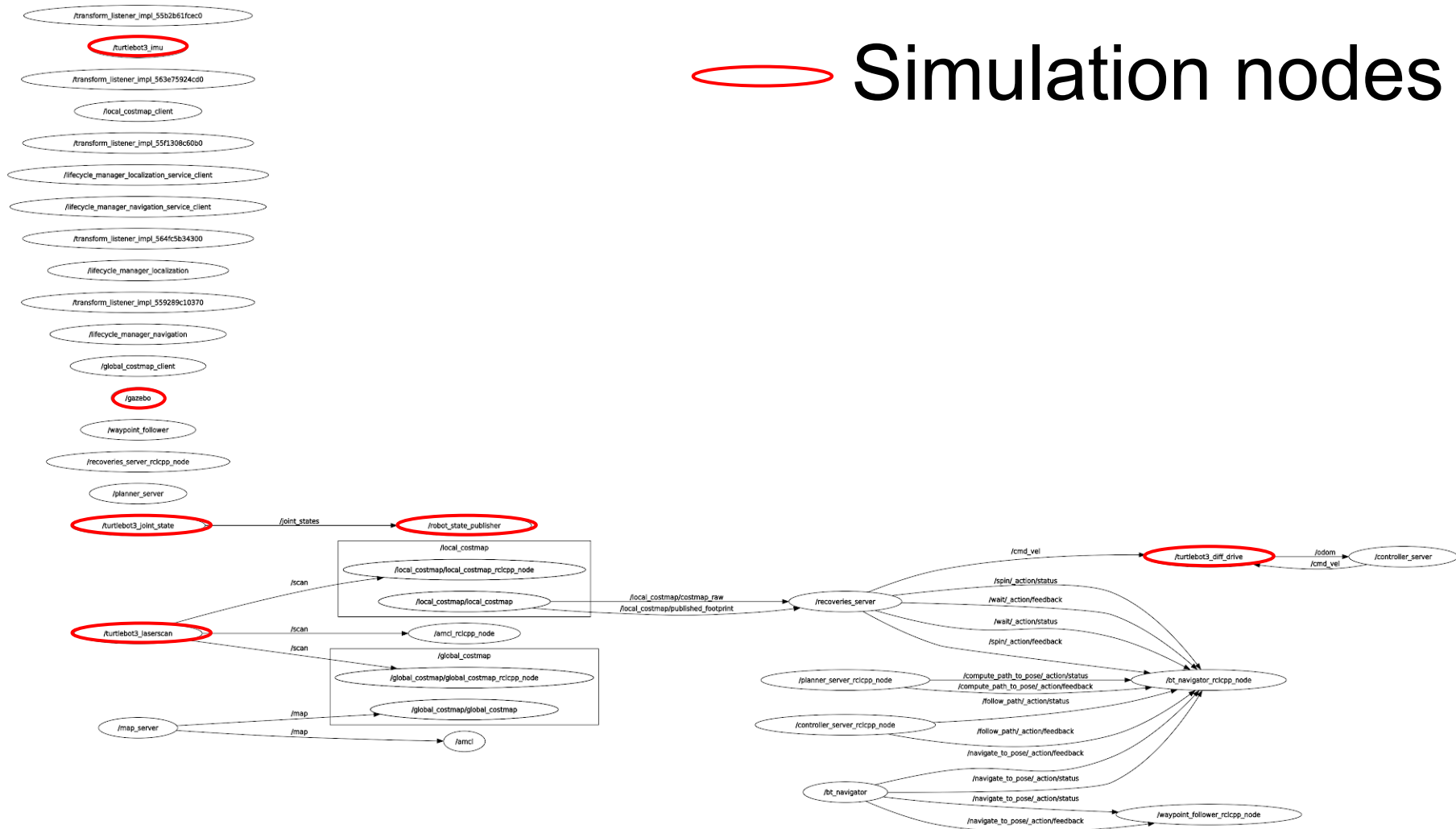
- `joint_states` (`sensor_msgs/msg/JointState`) - The joint state updates to the robot poses. The `RobotStatePublisher` class takes these updates, does transformations, and then publishes the results on the tf2 topics.

/robot_state_publisher

Published Topics

- `robot_description` (`std_msgs/msg/String`) - The description of the robot URDF as a string. Republishes the value set in the `robot_description` parameter, which is useful for getting informed of dynamic changes to the URDF.
- `tf` (`tf2_msgs/msg/TFMessage`) - The transforms corresponding to the movable joints of the robot.
- `tf_static` (`tf2_msgs/msg/TFMessage`) - The transforms corresponding to the static joints of the robot.

Simulation nodes



Navigation launch

turtlebot3_navigation2 / navigation2.launch.py

→ nav2_bringup / bringup_launch.py

→ nav2_bringup / localization_launch.py

→ nav2_bringup / navigation_launch.py

→ rviz2 -d nav2_bringup/rviz/nav2_default_view.rviz

Navigation launch: localization

nav2_bringup / localization_launch.py



→ nav2_map_server / map_server

→ nav2_amcl / amcl

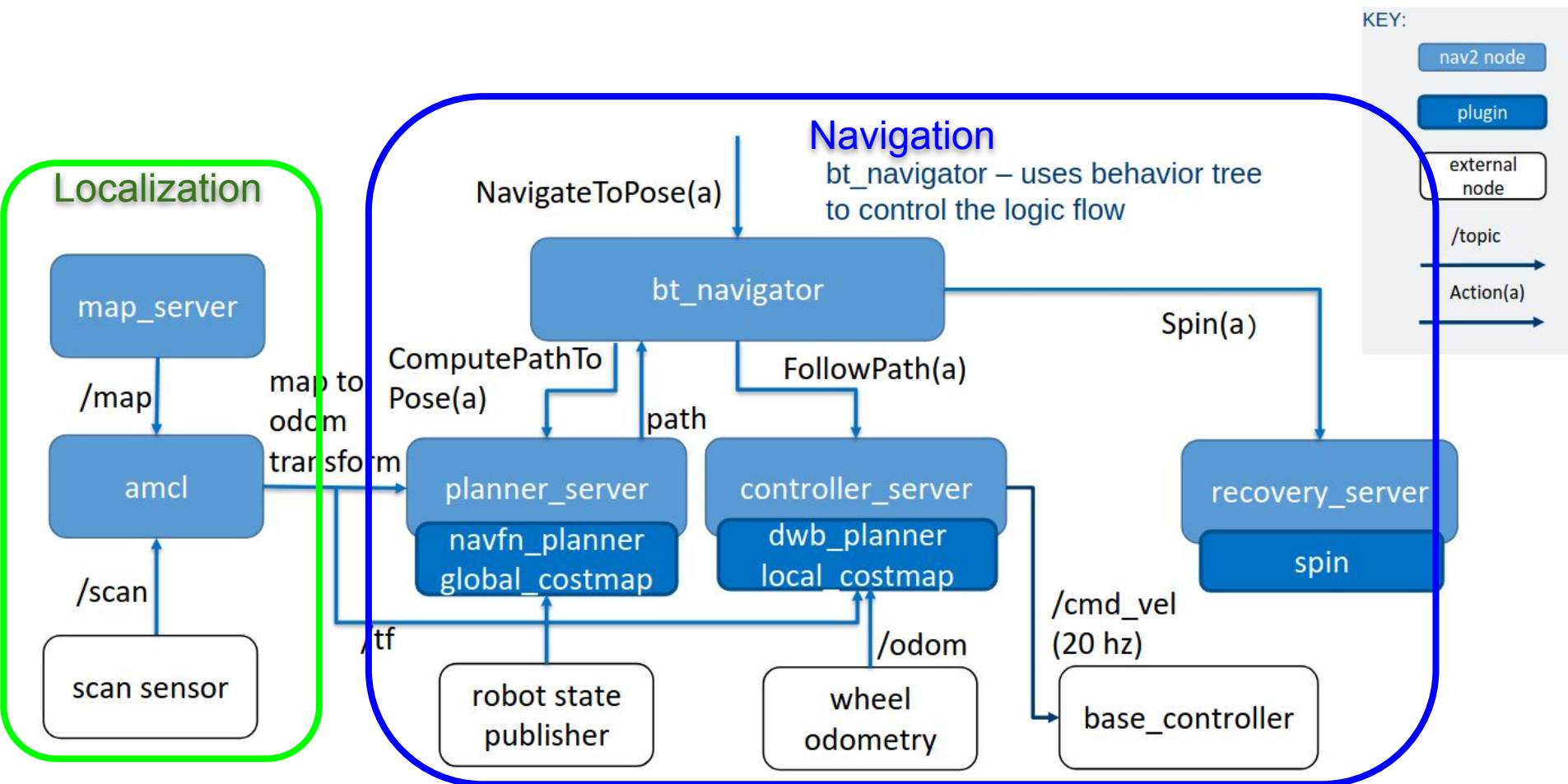
→ nav2_lifecycle_manager / lifecycle_manager

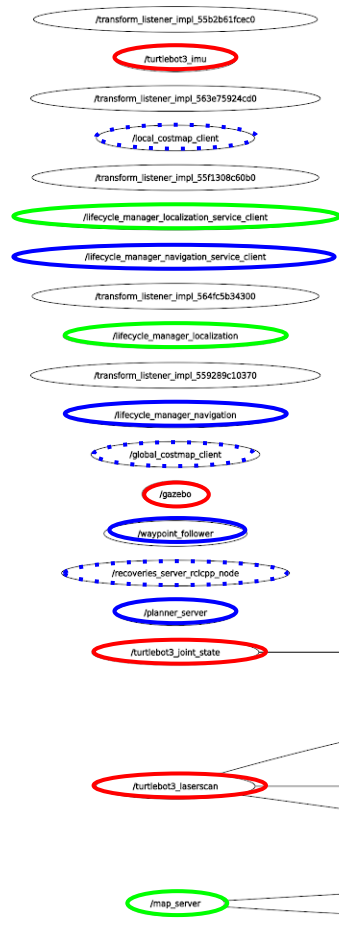
Navigation launch: navigation

nav2_bringup / navigation_launch.py

- nav2_controller / controller_server
- nav2_planner / planner_server
- nav2_recoveries / recoveries_server
- nav2_bt_navigator / bt_navigator
- nav2_waypoint_follower / waypoint_follower
- nav2_lifecycle_manager / lifecycle_manager

ROS Navigation2 Architecture

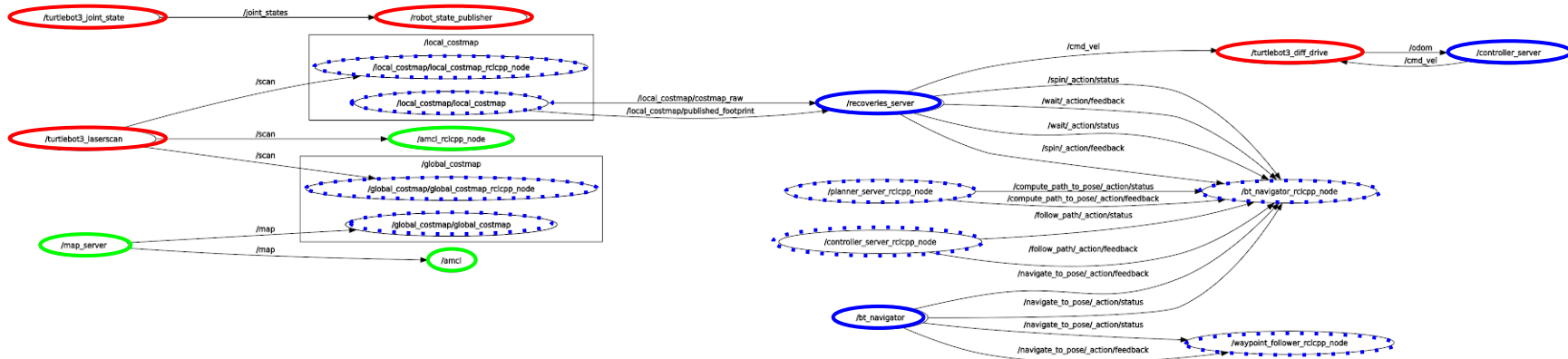




 Simulation nodes

 Localization nodes

 Navigation nodes



Navigation Concepts

- ROS 2
 - Action Server
 - Lifecycle Nodes
- Behavior Trees
- Navigation Servers
- State Estimation
- Environmental Representation

ROS2: Action Server

There are three forms of communication in ROS: topics, services, and actions.

Topic publishers broadcast to multiple subscribers, but communication is one-way.

Service clients send a request to a service server and get a response, but there is no information about the progress.

Similar to services, **action** clients send a request to an action server in order to achieve some goal and will get a result. Unlike services, **while the action is being performed an action server sends progress feedback to the client.**

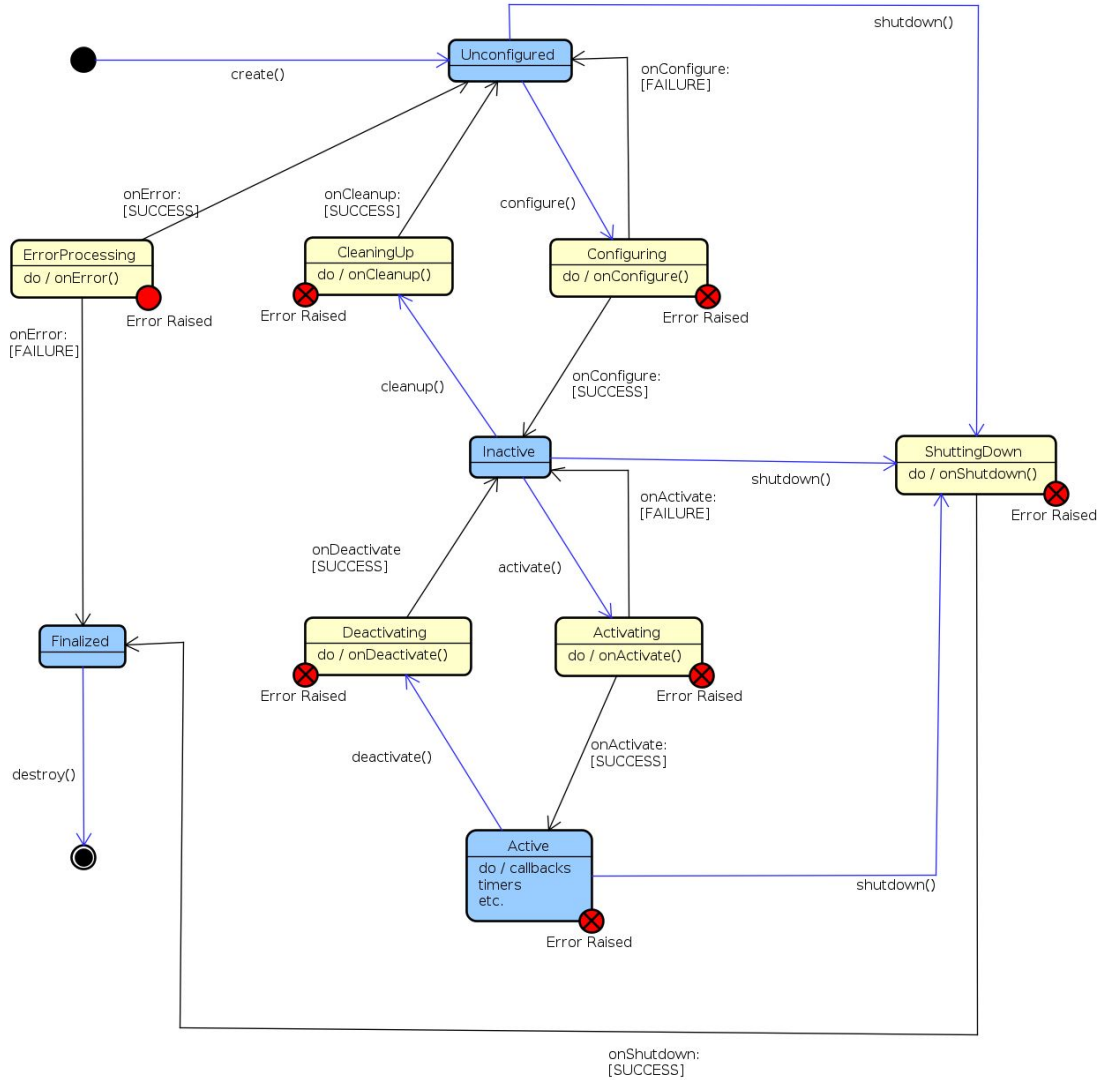
Actions are useful when **a response may take a significant length of time.** They allow a client to track the progress of a request, get the final outcome, and optionally cancel the request before it completes.

ROS2: Lifecycle Nodes

Lifecycle (or Managed, more correctly) nodes are ROS nodes that contain state machine transitions for bringup and teardown of servers.

This helps in deterministic behavior of ROS systems in startup and shutdown.

It also helps users structure their programs in reasonable ways for commercial uses and debugging.



Behavior Trees

A Behavior Tree (BT) is a way to structure the switching between different tasks in an autonomous agent, such as a robot or a virtual entity in a computer game.

BTs are a very efficient way of creating complex systems that are both modular and reactive.

These properties are crucial in many applications, which has led to the spread of BT from computer game programming to many branches of AI and Robotics.

Read Chapters 1-3 of “[Behavior Trees in Robotics and AI: An Introduction](#)”

[BehaviorTree CPP V3](#) is the behavior tree library of ROS2 Navigation.

Navigation Servers

Planners

Controllers

Recovery Behaviors

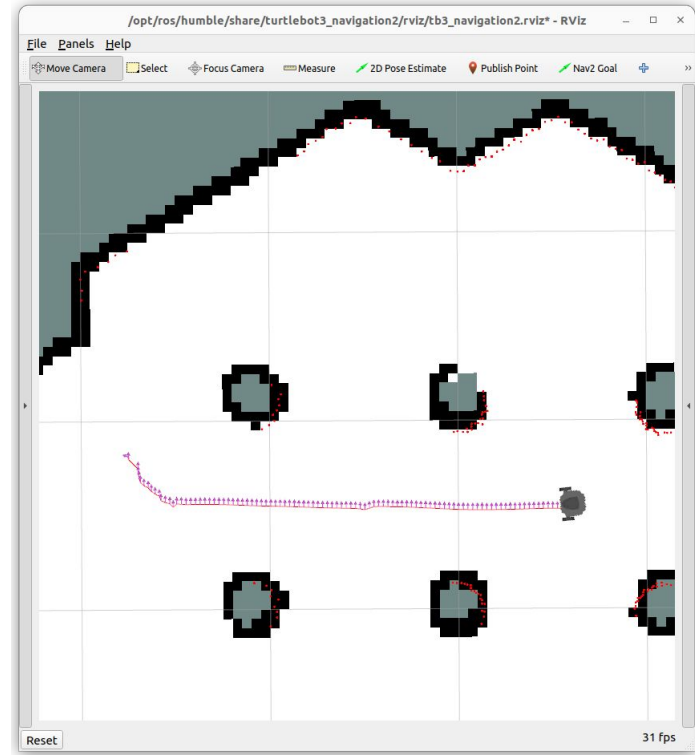
Waypoint Following

Planners

The task of a planner is to compute a path to complete some objective function.

The path can also be known as a route, depending on the nomenclature .

The general task in Nav2 for the planner is to compute a valid, and potentially optimal, path from the current pose to a goal pose.

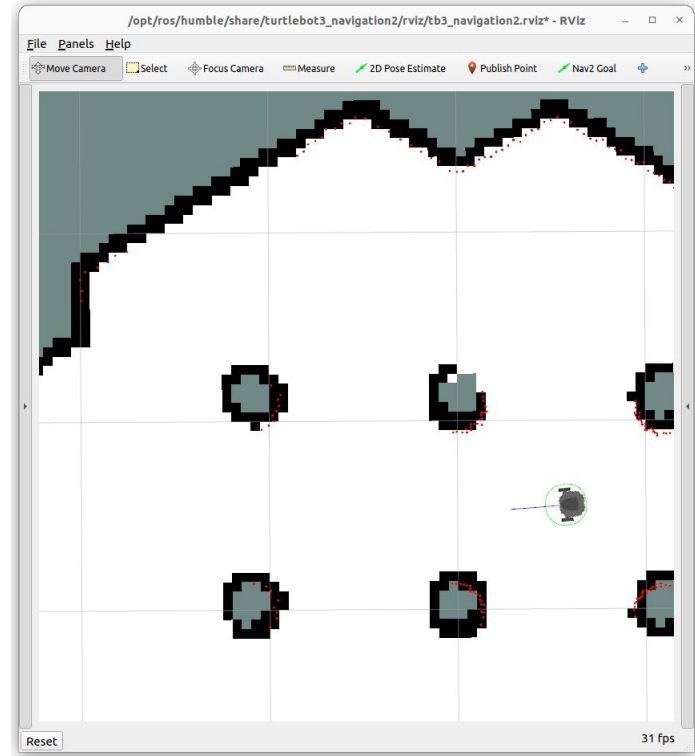


Controllers

Controllers, also known as local planners in ROS 1, are the way we follow the globally computed path.

Controllers can be written to:

- Follow a path
- Dock with a charging station using detectors in the odometric frame
- Board an elevator
- Interface with a tool



Recovery Behaviors

The goal of recoveries are to deal with unknown or failure conditions of the system and autonomously handle them.

An example would be if the robot was stuck due to dynamic obstacles or poor control. Backing up or spinning in place, if permissible, allow the robot to move from a poor location into free space it may navigate successfully.

Waypoint Following

The `nav2_waypoint_follower` contains a waypoint following program with a plugin interface for specific task executors.

This is useful if you need to go to a given location and complete a specific task like take a picture, pick up a box, or wait for user input.

It is a nice demo application for how to use Nav2 in a sample application.

State Estimation

You must, at minimum, build a [TF tree](#) that contains a full `map -> odom -> base_link -> [sensor frames]` for your robot.

TF2 are the time-variant transformation library in ROS 2 we use to represent and obtain time synchronized transformations.

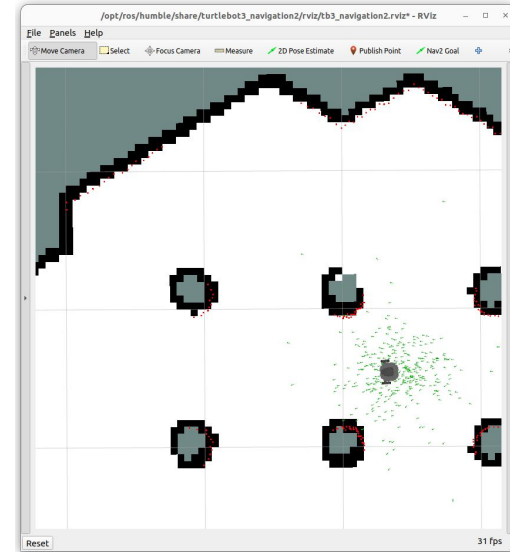
It is the job of the global positioning system (GPS, SLAM, Motion Capture) to, at minimum, provide the `map -> odom` transformation.

It is then the role of the odometry system to provide the `odom -> base_link` transformation.

The remainder of the transformations relative to `base_link` should be static and defined in your [URDF](#).

Localization

It is the job of the global positioning system (GPS, SLAM, Motion Capture) to provide the **map -> odom transformation**.



ROS provides [amcl](#) which is an [Adaptive Monte-Carlo Localization](#) technique based on a particle filter for localization of a static map.

It also provides [SLAM Toolbox](#) as the default SLAM algorithm for use to position and generate a static map.

Odometry

It is the role of the odometry system to provide the **odom -> base_link transformation**.

Odometry can come from many sources including LIDAR, RADAR, wheel encoders, VIO, and IMUs.

The goal of the odometry is to provide a smooth and continuous local frame based on robot motion.

The global positioning system will update the transformation relative to the global frame to account for the odometric drift.

Environmental Representation

The environmental representation is the way the robot perceives its environment.

It also acts as the central localization for various algorithms and data sources to combine their information into a single space.

This space is then used by the controllers, planners, and recoveries to compute their tasks safely and efficiently.

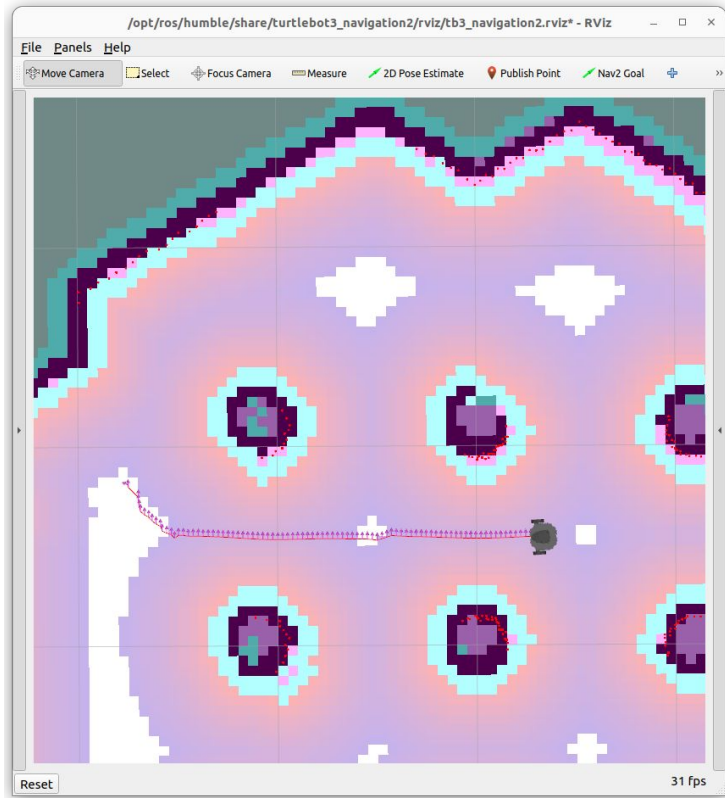
Costmaps and Layers

A costmap is a regular 2D grid of cells containing a cost from unknown, free, occupied, or inflated cost.

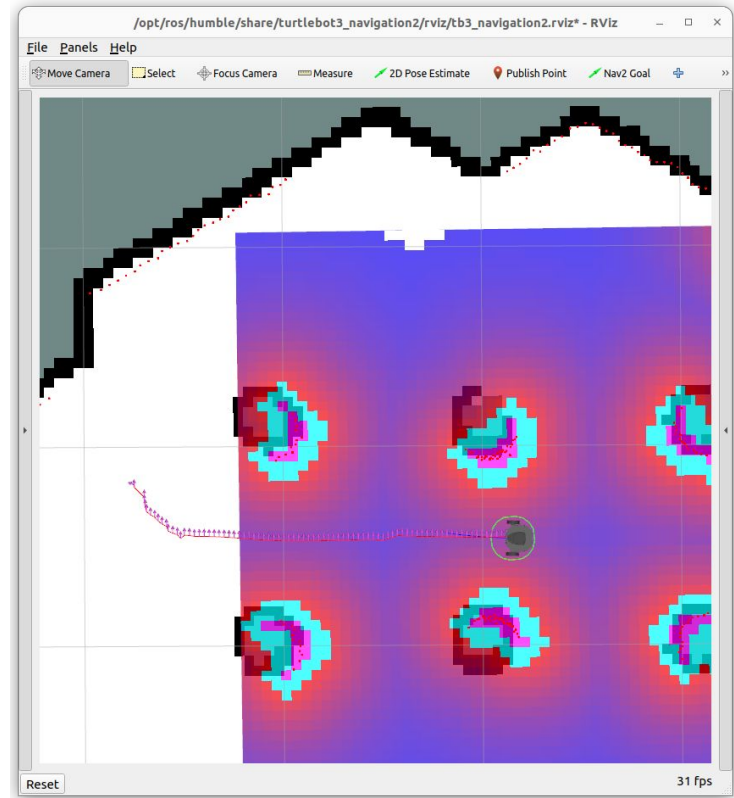
This costmap is then searched to compute a global plan or sampled to compute local control efforts.

Costmap layers can be created to detect and track obstacles in the scene for collision avoidance using camera or depth sensors.

Global costmap



Local costmap



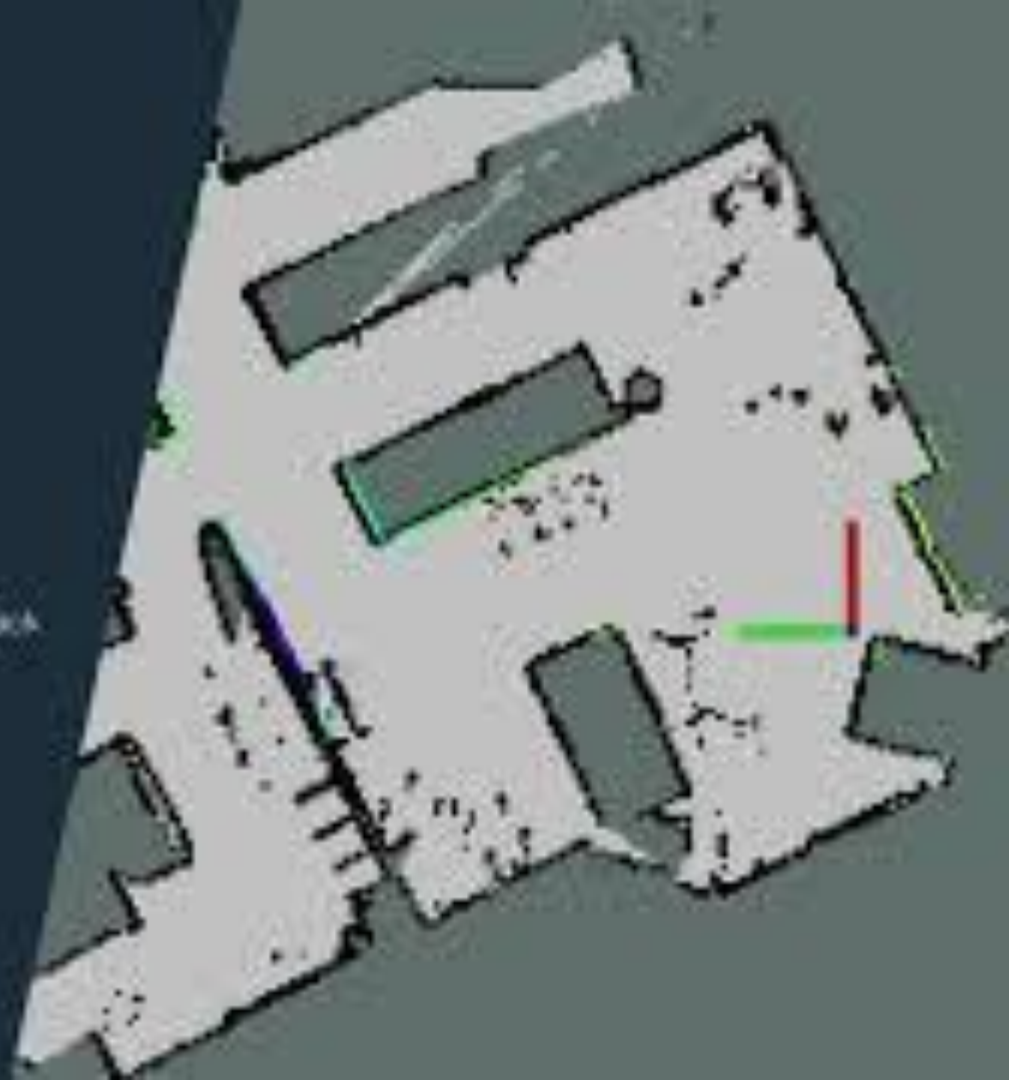
SAMSUNG

NAVIGATING BEYOND NAVIGATION

The Marathon 2: A Navigation System

A basic introduction to the new and improved ROS 2 Navigation Stack

Stewa Macenski, Samsung Research America



Navigation task: record and playback

Launch the Gazebo simulation of the TurtleBot 3 in the house.

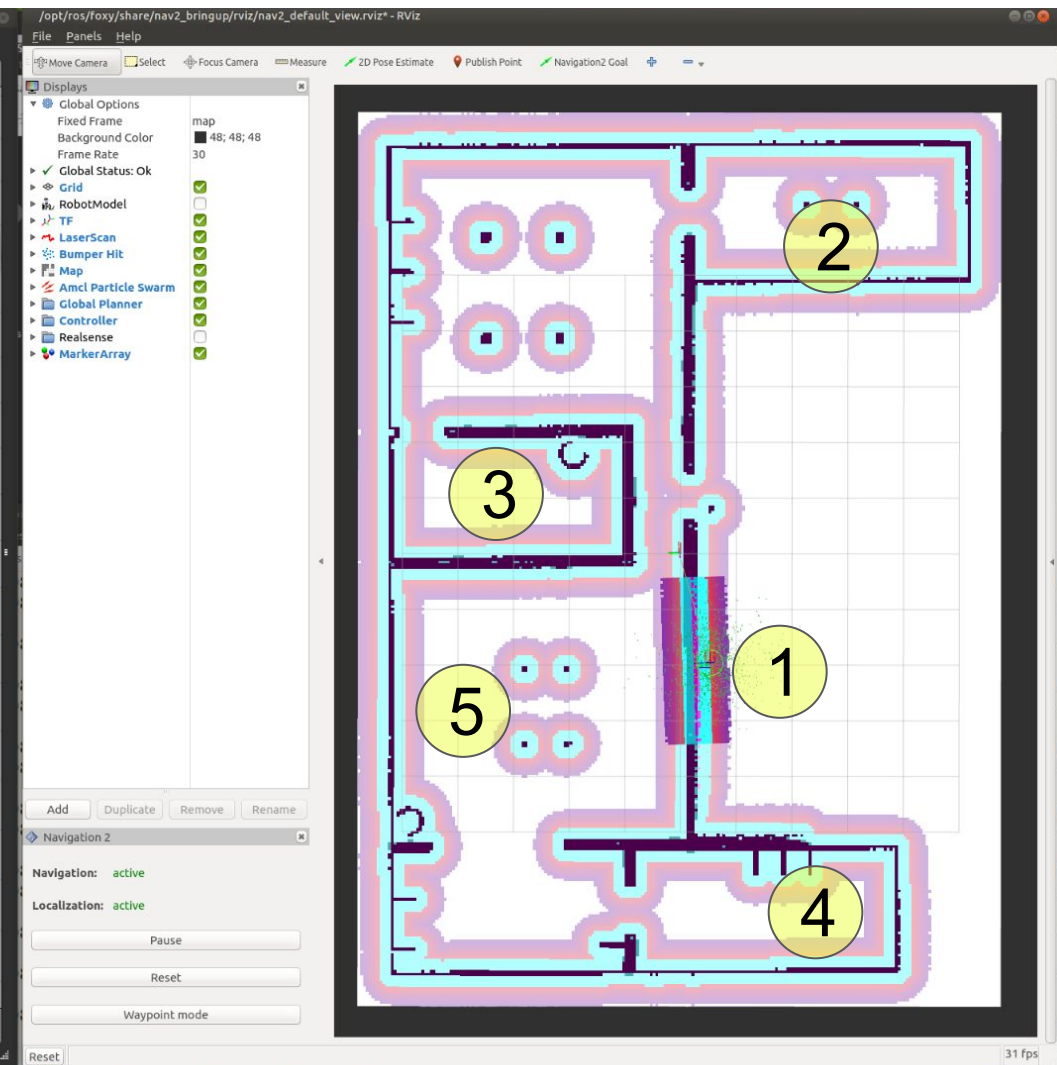
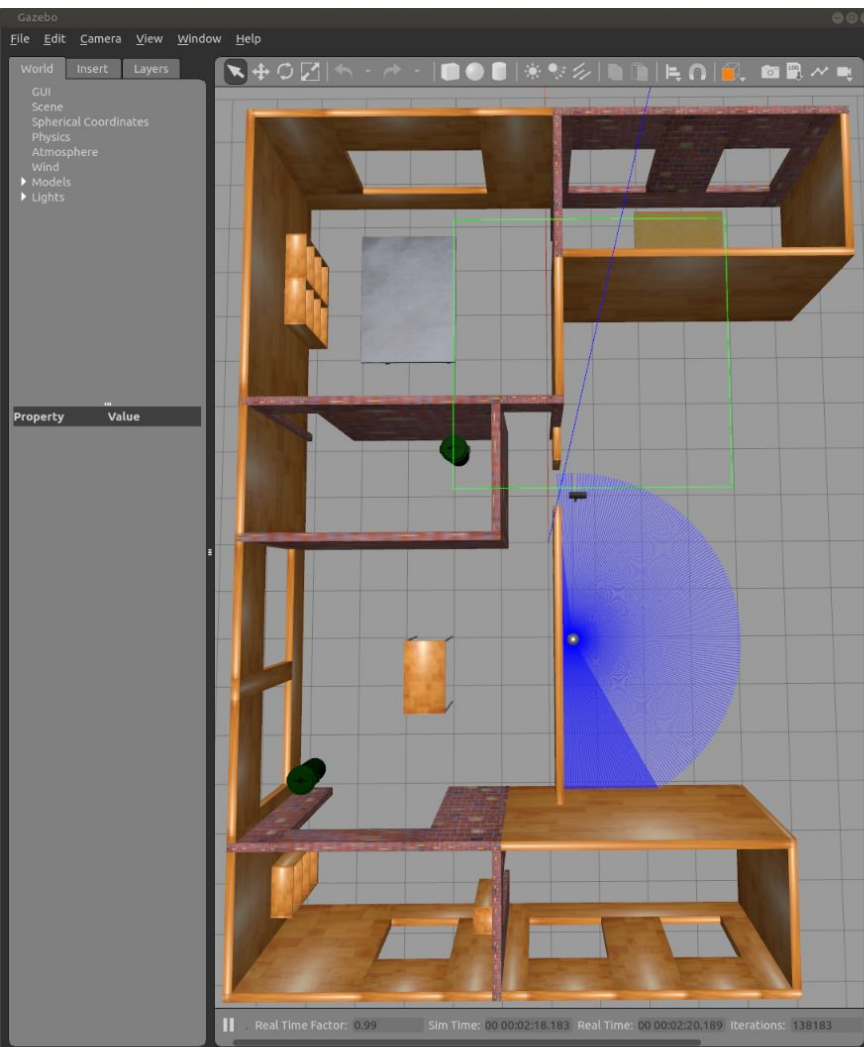
Record the following topics(*):

```
/clock /tf /tf_static /map /robot_description /scan /particle_cloud /plan  
/local_plan /waypoints /global_costmap/costmap /global_costmap/costmap_updates  
/local_costmap/costmap /local_costmap/costmap_updates  
/local_costmap/published_footprint /goal_pose /amcl_pose
```

Set the initial position of the robot (1) as shown in the next figure.

Navigate to the positions (2) to (5) as shown in the next figure.

(*) some of the topics may not have any message recorded



Verify and submit the recording to Aula Virtual

Verification:

```
rviz2 -d \  
    /opt/ros/humble/share/nav2_bringup/rviz/nav2_default_view.rviz  
  
ros2 bag play rosbag2_2025...
```

Patrolling task

Make a **ROS2 package** with a **ROS2 node** in **Python or C++** for sending a list of goals to the robot in the house.

The robot will navigate to each goal.

Upon arriving, the node will print a message and proceed to the next goal.

How to send goal

```
ros2 topic pub --once /goal_pose \  
  geometry_msgs/msg/PoseStamped \  
  "{header: {stamp: {}, frame_id: 'map'}, \  
    pose: {position: {x: -1, y: 3, z: 0.0}, \  
      orientation: {x: 0.0, y: 0.0, z: 0.0, w: 1.0}}} }"
```

GitHub versions

Version 1: send one goal

Version 2: send one goal and wait for completion
(odom pose does not change)

Version 3: send a sequence of goals

Version 4: wait for completion based on `amcl_pose`

Version 5: wait for completion based on `tf2`

Sub-versions

Version 1.0: publisher.cpp

Version 1.1: change topic class

Version 1.2: set data in message

Version 2.0: add subscriber

Version 2.1: change subscription topic to /odom

Version 2.2: analyze data in message

Version 2.3: set stopping condition

...

REMINDER: Acknowledge the use of AI

En els treballs de l'estudiantat, **l'ús literal de respostes generades per IA com si foren originals es pot considerar una acció fraudulenta**, segons l'article 11 de la [normativa d'avaluació de grau i màster de l'UJI](#).

If you completed any work with the aid of an AI tool,
you should always acknowledge the use.

One way this can be done is in a tabular form as shown below:

AI Tool Used	Prompt and output	How the output is used in the assignment

Patrolling with TurtleBot 2/3

Program the TurtleBot 2/3 for patrolling along a list of goals in the TD building.

You do not need to go beyond the WiFi range: the lab, the corridor, and the nearest part of the roundabout are fine.

If you want to go farther away, use the Ethernet cable and a laptop.